

ArrayStats (1-D integer arrays)

A class `ArrayStats` stores an integer array and provides several utility operations on that array.

```
public class ArrayStats {

    /** the array of integers (may contain positive, negative, or zero values) */
    private int[] data;

    /**
     * Constructs an ArrayStats object that stores a copy of arr.
     * Precondition: arr != null and arr.length > 0
     */
    public ArrayStats(int[] arr) { /* to be implemented */ }

    /**
     * Returns the number of even integers in data whose indices are in the
     * range start..end (inclusive).
     *
     * Preconditions: 0 <= start <= end <= data.length - 1
     */
    public int countEvensInRange(int start, int end) { /* part (a) */ }

    /**
     * Removes all occurrences of value from data by shifting remaining
     * elements left to preserve relative order. Vacated positions at the
     * end of data are filled with 0. Returns the number of elements removed.
     *
     * Example: data initially [3, 2, 2, 5, 2] and value = 2
     * After call: data becomes [3, 5, 0, 0, 0] and method returns 3.
     */
    public int removeAll(int value) { /* part (b) */ }

    /**
     * Returns the length (an int >= 1) of the longest contiguous run of
     * equal values in data. For example, for [1,1,2,2,2,3,3,2] the method
     * returns 3 (the run of three 2's).
     *
     * Precondition: data.length > 0
     */
    public int longestRun() { /* part (c) */ }

    /* There may be other methods, constructors, and instance variables. */
}
```

Part (a) — countEvensInRange

Write `countEvensInRange`. The method should iterate only over indices `start` through `end` and return how many entries in that subrange are even.

Example executions:

```
data = [4, 7, 0, -3, 8, 9]
```

Call	Return value	Explanation
<code>countEvensInRange(0, 2)</code>	2	4 and 0 are even
<code>countEvensInRange(2, 5)</code>	1	0 is even; -3, 8, 9 → note: if end were 4 then 8 would also be counted

```
public int countEvensInRange(int start, int end)
```

Part (b) — removeAll

Write `removeAll` so that it:

- removes every occurrence of `value` by shifting other elements to the left (keep original order),
- fills trailing positions with `0`,
- returns the number of removed elements.

You must perform the modification in-place (i.e., update `data`).

Example executions

1. `data` before: `[3, 2, 2, 5, 2]`, call `removeAll(2)`.
`data` after: `[3, 5, 0, 0, 0]`; method returns `3`.
2. `data` before: `[1, 1, 1]`, call `removeAll(1)`.
`data` after: `[0, 0, 0]`; method returns `3`.
3. `data` before: `[4, 5, 6]`, call `removeAll(7)`.
`data` after: `[4, 5, 6]`; method returns `0`.

```
public int removeAll(int value)
```

Part (c) — longestRun

Write `longestRun` to scan `data` and return the length of the longest contiguous subsequence where all values are equal. Do not modify `data`.

Example executions:

data	return
<code>[1,1,2,2,2,3,3,2]</code>	<code>3</code>
<code>[5]</code>	<code>1</code>
<code>[2,2,2,2]</code>	<code>4</code>
<code>[1,2,3,4]</code>	<code>1</code>

```
public int removeAll(int value)
```